

Lab 3 – Architecture Selection & Justification

Problem Statement #23 – Hyperlocal Courier Dispatch & Tracking Engine

Name: Darshana S

SRN: PES1UG24CS913

Section: 5 J

Architecture Selection:

I chose **Microservices Architecture** for the Hyperlocal Courier Dispatch & Tracking Engine.

Justification:

Architectural Choice:

Microservices Architecture. The system is decomposed into six independently deployable services — Live Tracking Service, Dispatch & Matching Service, Rider Dispatch Notification Service, Route Optimization Service, Delivery Completion Service, and OTP Verification Service — each owning a distinct responsibility from the Lab 1 requirements table and communicating through well-defined interfaces (ActiveRiderLocations, AssignedRider, RouteData, JobHandoff, OTPValidation), rather than sharing a single codebase or database. Delivery Completion Service owns FR-005's "mark delivery as Delivered" step and calls OTP Verification Service, a stateless validator, to check the code before allowing completion — keeping OTP validation logic separate from the workflow state that tracks whether a delivery is done.

Two Reasons:

Reason 1 — Independent Scaling for Very Different Load Patterns:

Live Tracking must continuously ingest GPS telemetry from every active rider and push updates to senders (FR-002, NFR-001 — under 2-second latency), which is a sustained, high-frequency load. Dispatch & Matching, by contrast, only activates in short bursts when a new pickup request comes in (FR-001). A layered or client-server design would force both workloads through the same application tier, so scaling for tracking's constant load would over-provision the rarely-busy matching logic. As independent microservices, each can be scaled to its own traffic pattern — e.g., running more Live Tracking instances during peak delivery hours without touching Dispatch & Matching capacity.

Reason 2 — Fault Isolation Matches the Stated Uptime Requirement:

NFR-002 requires the dispatch-matching service specifically to maintain 99.5% uptime. In a monolithic or tightly layered architecture, a failure or slow query in a less critical feature (e.g., Route Optimization for multi-stop parcels, FR-004) could exhaust shared resources and drag down the core matching path along with it. With microservices, Route Optimization or OTP Verification can degrade or fail independently without taking down Dispatch & Matching or Live Tracking, making the 99.5% target realistically achievable for the service it actually applies to.

Security Advantage:

Each microservice enforces its own access boundary and authenticates requests independently (e.g., via a per-service API gateway or token scope). This means a compromise of one service — say, Route Optimization, which only needs stop coordinates — does not automatically expose the OTP Verification Service's recipient codes or the Live Tracking Service's raw rider location data, since those live in separate services with separate credentials and data stores. A single shared database in a monolithic design would not offer this same containment.

Performance Benefit:

Because Live Tracking is its own deployable unit, it can be optimized and horizontally scaled purely for low-latency, high-throughput location streaming — using lighter-weight infrastructure suited to frequent small updates — without being slowed down by heavier, less latency-sensitive operations like Route Optimization's multi-stop calculations. This directly supports NFR-001's under-2-second telemetry latency requirement, since the tracking path is never queued behind unrelated, slower work.